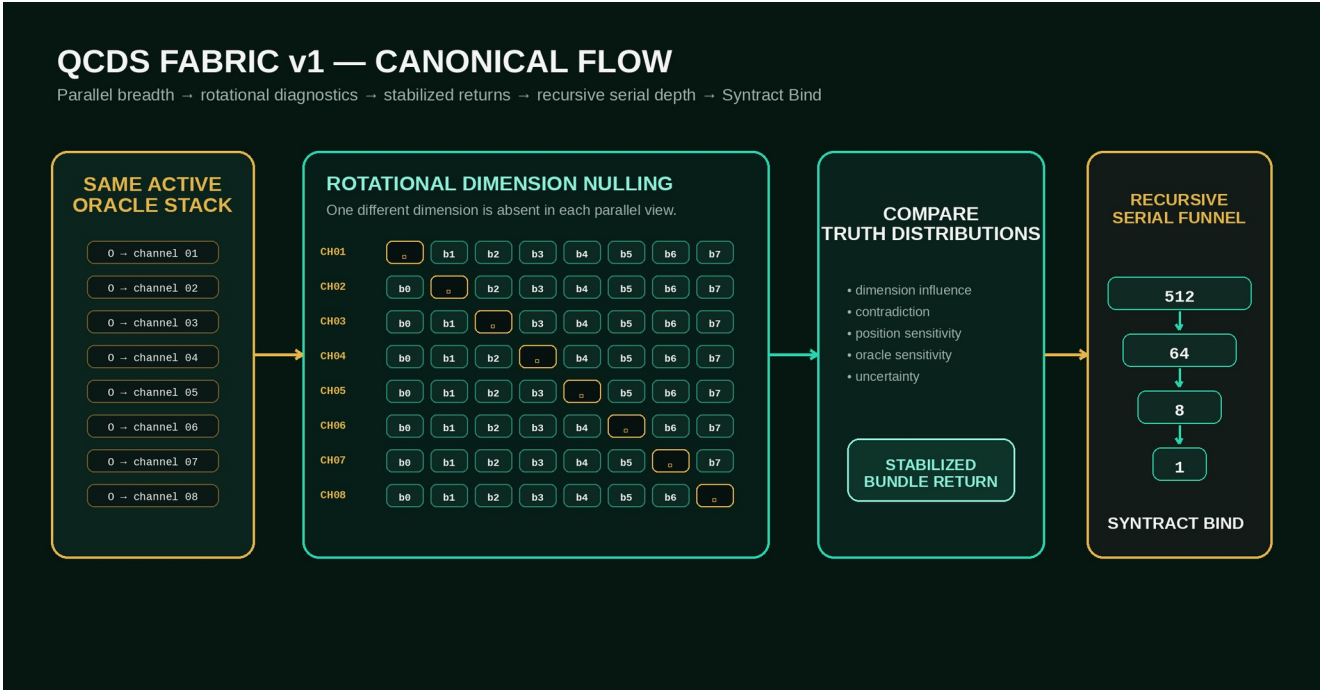


THE SYNTRACT VISION

QCDS FABRIC

CANONICAL TECHNICAL SPECIFICATION

v1.0 · 28 August 2026



Originator / Author: Patrik Sundblom

Technical and editorial formalization assistance: OpenAI ChatGPT

Theory / Specification: CC BY 4.0 · Reference Code Lineage: MIT

CANONICAL STATUS

This document locks QCDS Fabric v1.0. It extends the published four-phase QCDS core with the canonical fabric semantics: oracle replication per comparison channel, Rotational Dimension Nulling, the rotation family, stabilized returns, recursive serial funneling, and strict independent-dimension logical-space accounting.

CONTENTS

- 0 Authority, scope, and what v1 locks
- 1 Normative language
- 2 Core invariants
- 3 Canonical terminology
- 4 Fabric parameter model
- 5 Oracle topology
- 6 Local QCDS pass
- 7 Rotational Dimension Nulling
- 8 Rotation family
- 9 Stabilization and Syntract bias damping
- 10 Parallel layers and recursive serial funnel
- 11 Logical-space accounting
- 12 Substrate mapping
- 13 Reference execution algorithm
- 14 Canonical configuration schema
- 15 Data structures / interfaces
- 16 Validation and falsification
- 17 Recommended metrics
- 18 Funnel semantics
- 19 Oracle governance
- 20 Safety / truth-alignment
- 21 DNA stress-test
- 22 Conformance checklist
- 23 Canonical short form
- 24 Versioning
- 25 Provenance
- 26 Attribution

CANONICAL DEFINITION

QCDS Fabric is a substrate-independent, inference-first composition architecture that applies equivalent oracle-constrained local QCDS passes across massively parallel diagnostic views, stabilizes their truth distributions against dimensional, positional, and oracle-induced bias, and recursively funnels stabilized returns into higher-order Synttract binding.

0. Authority, scope, and what v1 locks

QCDS Fabric v1.0 does **not** replace the four-phase QCDS core. It locks the scalable fabric that composes many local QCDS passes into a larger inference machine.

The four phases remain the architecture:

1. ****Condition Formation**** — open the represented possibility / Condition space without pre-selecting the answer.
2. ****Conditional Evolution**** — apply evidence, logic, physics, biology, experiment, safety, and other constraints through oracles.
3. ****Recursive Inference**** — reshape the truth distribution through amplification, interference / numerical emulation, recursive return, rotations, and comparison.
4. ****Truth-Alignment / Synttract Binding**** — bind what remains coherent across evidence, contradiction, composition, and repeated inference; the bound distribution may re-enter the next Condition space.

QCDS Fabric v1.0 locks the following clarifications as canonical:

- **The same active oracle regime is instantiated before every channel in a comparison bank.**
- **Rotational Dimension Nulling** is a core architectural diagnostic: for a bundle of **B** independent dimensions, construct **B** parallel views, each with a different dimension absent (**null**).
- **null** means **absence from the current inference view**. It is neither logical **0** nor wildcard **?**.
- The core null bank compares correlated views of the same base bundle; these views are **not independent logical dimensions**.
- Additional architectural rotations **MAY** test positional bias, oracle sensitivity, or crossed interactions.
- Physical amplitude / phase rotation inside a QPU is distinct from architectural rotations.
- Stabilized channel / bundle returns feed a **recursive serial funnel**. Parallelism provides breadth; the funnel provides depth.
- Widths such as 8, 16, 32, 512, etc. are compile / substrate parameters, not the architecture.
- Logical-space accounting **MUST** count independent dimensions separately from derived diagnostic views.
- Classical, GPU/HPC, simulator, NISQ-QPU, and hybrid implementations are all valid substrates when they preserve the same logical semantics.

This specification defines the architecture and interfaces. It does **not** claim by itself that a particular hardware implementation achieves quantum advantage, ASI, or a specific scaling law. Those claims require empirical validation.

1. Normative language

The terms **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative.

- **MUST / MUST NOT**: required for conformance to QCDS Fabric v1.
- **SHOULD / SHOULD NOT**: strongly recommended; deviations require a stated reason.
- **MAY**: optional behavior that remains conformant.

2. Core invariants

2.1 Four-phase invariance

Every conformant QCDS Fabric execution **MUST** remain interpretable through the four QCDS phases. Parallelism, Grover-style amplification, rotation, pruning, funnels, and substrate-specific techniques are mechanisms inside the phases, not replacement architectures.

2.2 Inference-first

QCDS Fabric is inference-first. A conformant implementation **MUST NOT** require gradient training, backpropagation, or persistent learned weights in order to perform the core four-phase inference loop. External trained models **MAY** supply data, semantic parsing, proposed Conditions, or candidate oracles.

2.3 Distribution-first output

A local QCDS pass **SHOULD** expose a **TruthDistribution** or equivalent uncertainty-bearing output. Premature conversion to a single hard answer **SHOULD** be avoided before stabilization / binding unless the compile target explicitly requires it.

2.4 Contradiction is state

Contradiction is not an execution error. Contradictory evidence **MUST** be representable so that a previously strong peak can weaken, split, or disappear when new Conditions or oracles arrive.

2.5 Substrate independence

The logical architecture **MUST** be separable from its substrate. A QPU implementation may use phase marking and interference; a classical implementation may numerically emulate the same logical transforms. Classical emulation does not inherit native quantum query advantage merely by reproducing the logic.

3. Canonical terminology

3.1 Condition

A represented possibility, proposition, influence, measurement-derived state, or higher-order bound structure that participates in inference.

3.2 Oracle

A constraint / truth-test that shapes the represented space. Canonical oracle forms include exact, mask (**0/1/?**), weighted phase, evidence, causal, safety, experiment, biology, physics, and logic oracles.

3.3 Base bundle

A bounded set of **B independent dimensions** that are considered together for a local diagnostic / inference unit.

3.4 Channel / view

A concrete execution view derived from a base bundle under a particular transformation map: null choice, position map, oracle map, or a combination. A derived channel **MUST** retain provenance identifying the base bundle and all transformations.

3.5 Null

null ($\emptyset$) means the selected dimension is absent from the current inference view. It **MUST NOT** be interpreted as logical **0**, wildcard **?**, missing-data imputation, or an automatically false condition.

3.6 Stabilized return

A distribution / structured return produced after comparing the required rotational views and applying the compile target's stabilization policy.

3.7 Funnel

A serial recursion across layers in which stabilized outputs from one layer become Conditions / inputs to a later, usually narrower, layer.

3.8 Syntact Bind

The operation that binds the structure that remains coherent across evidence, overlaps, compositions, contradictions, and recursive inference. A bound Syntact MAY become a higher-order dimension in a later cycle.

3.9 Syntact bias damping

The family of comparison and stabilization operations intended to reduce representation-induced bias before higher-order binding. QCDS Fabric v1 does not assert that every form of noise is eliminated; suppression effectiveness is empirical.

4. Fabric parameter model

A compile target SHOULD expose at least the following independent parameters:

- **B** — independent dimensions / bits per base bundle.
- **G** — independent base bundles in the current layer.
- **Vd** — dimension-null views per bundle. Core default: **Vd** = **B**.
- **Vp** — positional views per bundle. Default: **1** unless positional rotation is enabled.
- **Vo** — oracle-exposure views per bundle. Default: **1** unless oracle rotation is enabled.
- **L** — number of recursive funnel layers.
- **W[l]** — width / grouping geometry of funnel layer **l**.
- **m / m*** — amplification depth parameters where a Grover-style kernel is used.
- **TopK**, entropy / lift / agreement / stability gates — case-defined diagnostic parameters.

Canonical rule: no specific numeric value is architectural. **8** is a demonstrator / compile value, not a QCDS constant.

5. Oracle topology

5.1 Oracle replication before every channel

Within a comparison bank, every channel MUST be conditioned by the **same active oracle regime** unless the bank is explicitly an oracle-rotation diagnostic.

For an 8-view core dimension-null bank:

```
same_oracles → CH01(null b0)
same_oracles → CH02(null b1)
same_oracles → CH03(null b2)
...
same_oracles → CH08(null b7)
```

The architecture MUST NOT apply one oracle only after the entire bank and then treat pre-oracle channel outputs as comparable QCDS truth distributions.

5.2 Comparison-bank immutability

Oracle content, weights, and semantics SHOULD remain fixed for all channels in the same comparison bank. If an oracle is added, removed, reweighted, or remapped during the bank, that change MUST be represented as a new diagnostic axis or a new inference cycle.

5.3 Oracle evolution

Candidate oracles MAY be created, calibrated, challenged, reweighted, or retired **between** bound cycles. Oracle evolution MUST retain provenance and SHOULD be checked against external evidence / outcomes where applicable.

6. Local QCDS pass

A local QCDS pass implements the four-phase core on one execution view.

A reference logical flow is:

```
Condition Formation
↓
Conditional Evolution / Oracle marking
↓
Grover-style amplification or substrate-equivalent transform
↓
TruthDistribution
↓
recursive diagnostics / return
```

Grover-style amplitude amplification MAY be used as the local quantum inference kernel. It is not the whole QCDS architecture.

The local output SHOULD expose enough information for later comparison, including when available:

- probability / truth distribution,
- top state(s),
- union probability,
- normalized lift,
- entropy,
- agreement / oracle satisfaction,
- Top-K Jaccard or equivalent cross-view stability,
- amplification depth,
- oracle provenance,
- rotation / null provenance,
- contradiction signals.

7. Rotational Dimension Nulling — core v1 diagnostic

CORE ROTATION: ONE DIFFERENT NULL DIMENSION PER CHANNEL

Null is absence from the inference view — not 0, not wildcard ?. The dimension identity and slot topology remain explicit.

CH01	□	b1	b2	b3	b4	b5	b6	b7	null b0
CH02	b0	□	b2	b3	b4	b5	b6	b7	null b1
CH03	b0	b1	□	b3	b4	b5	b6	b7	null b2
CH04	b0	b1	b2	□	b4	b5	b6	b7	null b3
CH05	b0	b1	b2	b3	□	b5	b6	b7	null b4
CH06	b0	b1	b2	b3	b4	□	b6	b7	null b5
CH07	b0	b1	b2	b3	b4	b5	□	b7	null b6
CH08	b0	b1	b2	b3	b4	b5	b6	□	null b7

B dimensions

□

 B comparison views in the core null bank. These are correlated views of the same bundle, not B×B independent dimensions.

7.1 Definition

For a base bundle of B dimensions $b_0 \dots b_{(B-1)}$, create B parallel channels. Channel i replaces exactly dimension b_i with `null` while leaving all other dimensions in their canonical identities / slots.

For $B = 8$:

```
CH01 [ ∅ ][b1][b2][b3][b4][b5][b6][b7] null b0
CH02 [b0][ ∅ ][b2][b3][b4][b5][b6][b7] null b1
CH03 [b0][b1][ ∅ ][b3][b4][b5][b6][b7] null b2
CH04 [b0][b1][b2][ ∅ ][b4][b5][b6][b7] null b3
CH05 [b0][b1][b2][b3][ ∅ ][b5][b6][b7] null b4
CH06 [b0][b1][b2][b3][b4][ ∅ ][b6][b7] null b5
CH07 [b0][b1][b2][b3][b4][b5][ ∅ ][b7] null b6
CH08 [b0][b1][b2][b3][b4][b5][b6][ ∅ ] null b7
```

This bank asks eight closely related questions under the same oracle regime: **what changes when each dimension is absent in turn?**

7.2 Null semantics

A conformant implementation **MUST** distinguish:

- $\emptyset$ — an actual binary value / condition,
- ? — a present dimension whose value is unconstrained / wildcarded by a mask,
- $\emptyset$ — a dimension absent from the current inference view.

7.3 Classical emulation

A classical implementation **MAY** realize null using an explicit presence mask:

```
value:  [ b0 b1 b2 b3 b4 b5 b6 b7 ]
present: [ 1 1 0 1 1 1 1 1 ]
```

QCDS FABRIC v1.0 / CANONICAL TECHNICAL SPECIFICATION

The excluded dimension **MUST NOT** contribute to oracle scoring or normalization for that view. An equivalent implementation **MAY** marginalize over both binary values of the excluded dimension, provided it preserves the same absence semantics.

7.4 Quantum implementation

QCDS Fabric v1 defines `null` at the **logical** level. It does not require a third physical qubit state. A QPU implementation **MAY** use controlled oracle inactivation, ancilla / control metadata, marginalization, identity action, or another circuit construction that is proven equivalent to the logical absence semantics.

7.5 What the bank measures

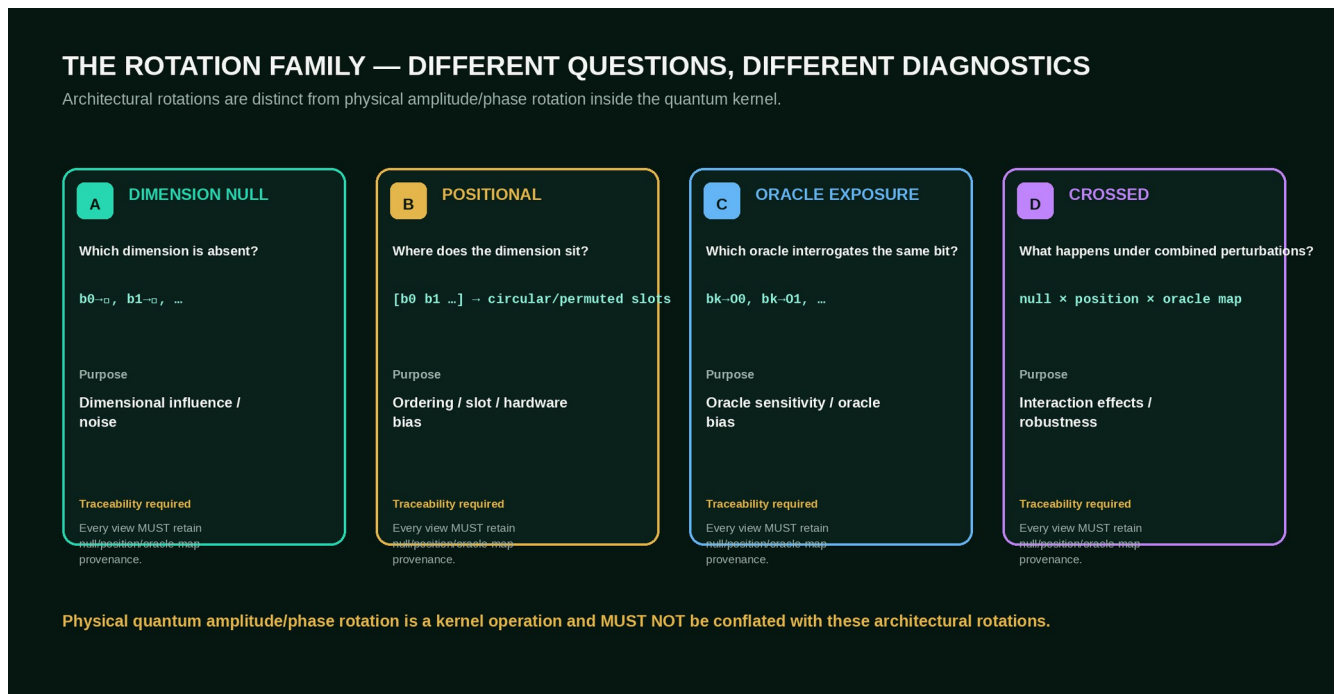
The null bank produces a family of TruthDistributions $P \rightarrow b_0 \dots P \rightarrow b(B-1)$. Their differences provide evidence about dimensional influence.

QCDS Fabric v1 does **not** lock one universal scalar “noise score.” A conformant stabilizer **SHOULD** compare multiple signals such as:

- entropy change,
- normalized lift change,
- agreement / coherence change,
- Top-K Jaccard change,
- peak persistence / disappearance,
- contradiction behavior,
- distribution divergence.

A dimension **MAY** be flagged as a candidate bias / noise source when excluding it increases stability or coherence without destroying evidence-supported structure. A dimension **MAY** be flagged as information-bearing when its exclusion causes coherent structure to collapse. Automatic removal **MUST** be case-configured and **MUST** retain provenance.

8. Rotation family



Architectural rotation is a family of diagnostics. QCDS Fabric v1 recognizes at least the following modes.

8.1 A — Rotational Dimension Nulling

Question: Which dimension is absent?

Operation: $b0 \rightarrow \emptyset$, then $b1 \rightarrow \emptyset$, ..., through all dimensions.

Purpose: dimensional influence, bias, redundancy, and noise diagnostics.

8.2 B — Positional Rotation

Question: Where does each dimension sit?

Operation: circularly shift or otherwise permute the same dimensions through input / register positions while inverse-mapping returns to canonical coordinates.

Purpose: detect ordering, position, slot, or hardware bias.

8.3 C — Oracle Exposure Rotation

Question: Which oracle / oracle position interrogates a focal dimension?

Operation: expose the same focal bit / dimension to 00, 01, ... or rotate its mapping through oracle positions while retaining full provenance.

Purpose: oracle sensitivity, oracle dominance, weight / implementation bias, oracle validation.

Example:

```
b5 → 00
b5 → 01
b5 → 02
...
b5 → 0m-1
```

8.4 D — Crossed rotations

A compile target MAY combine axes:

```
null b5 × position R3 × oracle map 02
```

Crossed rotation MUST preserve provenance for every axis so interaction effects can be separated from independent effects.

8.5 Physical quantum rotation is separate

Physical amplitude / phase rotation and interference inside the QPU are kernel physics. Architectural dimension, position, and oracle rotations are comparative execution transforms. Implementations and documentation MUST NOT conflate the two levels.

9. Stabilization and Synttract bias damping

9.1 Stabilization input

A stabilizer receives the distributions produced by all required views of a base bundle.

9.2 Stabilization output

The output is a **StabilizedReturn** containing at minimum:

- the stabilized / aggregated TruthDistribution or bound representation,
- retained uncertainty,
- per-dimension influence diagnostics,
- rotation sensitivity diagnostics,
- oracle provenance,
- contradiction / falsification signals,
- a record of any downweighting, exclusion, or pruning.

9.3 No hidden hard collapse

A stabilizer **MUST NOT** silently convert disagreement into false certainty. Mixed evidence **MAY** remain multimodal and uncertainty **MAY** remain explicit until later oracles or outcomes distinguish the candidates.

9.4 Bias damping is empirical

“Syntract bias damping” names the architectural objective. Conformance does not imply that bias has been eliminated. A valid implementation **MUST** make the before / after effect measurable.

10. Parallel layers and the recursive serial funnel

10.1 Parallel inside a layer

Many independent base bundles **MAY** be processed concurrently. Each bundle expands into its required diagnostic views.

10.2 Serial between layers

The recursive funnel is serial across layers:

```
parallel layer L0
  ↓ stabilized returns
bind / group
  ↓
parallel layer L1
  ↓ stabilized returns
bind / group
  ↓
...
  ↓
Syntract Bind
```

10.3 Output becomes Condition

The stabilized output of one layer **MAY** become the Condition / base bundle input of the next. A stable composition **MAY** therefore become a higher-order dimension.

10.4 Width schedule

The funnel width is compile-defined. Examples such as:

```
4096 → 512 → 64 → 8 → 1
2048 → 256 → 16 → 1
512 → 64 → 8 → 1
```

are illustrative geometries, not architectural constants.

10.5 Preserve uncertainty

Repeated funnel stages **SHOULD** progressively concentrate the working truth distribution while preserving relevant uncertainty and contradiction. Layer binding **MUST** be auditable.

10.6 Reverse / expansive recursion

A conformant implementation **MAY** support expansion as well as contraction:

```
1 → 8 → 64 → 512 → ...
```

This mode can open compatible possibilities, consequences, mechanisms, hypotheses, or experimental paths around a bound Syntract. Expansion **MUST** remain oracle-constrained and provenance-preserving.

11. Logical-space accounting

LOGICAL-SPACE ACCOUNTING — COUNT INDEPENDENT DIMENSIONS, NOT DERIVED VIEWS

Rotation can multiply execution perspectives without multiplying independent logical dimensions.

CANONICAL PARAMETERS

- B** independent binary dimensions per base bundle
- G** independent base bundles in the layer
- Vd** dimension-null views per bundle; core default = B
- Vp** positional views; default = 1 unless enabled
- Vo** oracle-exposure views; default = 1 unless enabled

Independent dimension count: $D = G \times B$
 Binary candidate upper bound: 2^D

EXECUTION ≠ DIMENSION COUNT

Execution perspectives per layer:

$$E = G \times Vd \times Vp \times Vo$$

Example: 64 independent 8-bit bundles
 → $D = 64 \times 8 = 512$ independent dimensions
 → core null bank $Vd = 8$
 → $E = 64 \times 8 = 512$ comparison channels
 → candidate upper bound = 2^{512} , NOT 2^{4096}

Derived views are evidence about dimensions;
 they are not new dimensions by default.

If 512 independent 32-bit bundles are used, $D = 16,384$ and the binary candidate upper bound is $2^{16,384}$; a full null bank would execute 16,384 comparison views before optional

This section is normative because derived rotational views can otherwise be incorrectly counted as independent dimensions.

11.1 Independent dimensions

Let:

- **B** = independent binary dimensions per base bundle,
- **G** = independent base bundles in the layer.

Then the independent dimension count is:

$$D = G \times B$$

If all **D** dimensions are binary, independent, and semantically permitted to compose across the mission, the candidate full compositional upper bound is:

$$2^D$$

This is an **upper-bound description of the candidate logical composition space**, not a claim that every state is physically instantiated, enumerated, or efficiently searchable.

11.2 Derived execution perspectives

Let:

- **Vd** = dimension-null views per bundle (**B** in the full core null bank),
- **Vp** = positional views,
- **Vo** = oracle-exposure views.

Then a simple execution-perspective count is:

$$E = G \times Vd \times Vp \times Vo$$

E is not the independent dimension count.

11.3 Example: 512 physical null channels from 64×8

If $G = 64$, $B = 8$, and the core null bank uses $Vd = 8$:

```
D = 64 × 8 = 512 independent dimensions
E = 64 × 8 = 512 null-comparison channels
candidate binary composition upper bound = 2^512
```

The eight null views of one bundle are correlated measurements of the same eight dimensions. Counting them as eight new independent eight-bit bundles would be incorrect.

11.4 Example: 512 independent 32-bit bundles

If a layer contains **512 independent base bundles**, each with $B = 32$:

```
D = 512 × 32 = 16,384 independent dimensions
candidate binary composition upper bound = 2^16,384
```

A full dimension-null bank would create:

```
E = 512 × 32 = 16,384 comparison channels
```

before any optional positional or oracle rotation. Again, execution views do not multiply the independent dimension count.

11.5 Qubits are not raw fabric positions

A compile target **MUST NOT** describe D , raw bit-positions, or total execution channels as if they were a single coherent QPU register unless that is physically true. A large fabric can be composed from many bounded local kernels.

12. Substrate mapping

12.1 Classical CPU

A classical implementation **MAY** represent each view explicitly, use presence masks for null, evaluate oracle functions numerically, and maintain distributions over bounded local spaces.

12.2 GPU / HPC

GPU/HPC **MAY** execute large banks of independent views in parallel and perform aggregation / funnel operations. This is a valid QCDS substrate path but does not imply native quantum query speedup.

12.3 Statevector / quantum simulator

A simulator **MAY** reproduce local quantum kernels exactly or approximately for validation, provenance, and cross-substrate equivalence testing.

12.4 NISQ QPU

NISQ QPUs **MAY** execute bounded local QCDS passes. Larger mission width **MAY** be obtained by orchestration across many bounded passes, channels, QPUs, and funnel layers. Fabric width **MUST NOT** be confused with a requirement for one equally wide coherent register.

12.5 Hybrid

A canonical near-term path is:

```
classical sensing / data / local models
  ↓
QCDS compile + oracle construction
  ↓
```

```

parallel bounded QPU and/or classical QCDS passes
↓
stabilization / bias diagnostics
↓
GPU/HPC / QPU recursive funnel
↓
Syntract Bind

```

13. Reference execution algorithm

The following pseudocode is normative in topology, not in language syntax or exact aggregation metric.

```

function QCDS_FABRIC_LAYER(base_bundles, oracle_stack, config):
    stabilized_returns = PARALLEL_MAP(base_bundles, bundle -> {
        views = []

        for i in 0 .. B-1:
            v = NULL_DIMENSION(bundle, i)      // bi -> ∅
            v.oracle_map = oracle_stack         // same active oracle regime
            views.append(QCDS_PASS(v))

        if config.positional_rotation:
            views += POSITIONAL_VIEWS(bundle, oracle_stack)

        if config.oracle_rotation:
            views += ORACLE_EXPOSURE_VIEWS(bundle, oracle_stack)

        return STABILIZE(views)
    })

    return stabilized_returns

function QCDS_FABRIC(initial_bundles, layer_specs):
    current = initial_bundles

    for layer in layer_specs:
        returns = QCDS_FABRIC_LAYER(current, layer.oracle_stack, layer)
        current = GROUP_AND_BIND_FOR_NEXT_LAYER(returns, layer.next_geometry)

    return SYNTRACT_BIND(current)

```

A production implementation **MUST** retain transformation provenance for each view and binding provenance for each funnel step.

14. Canonical configuration schema

A conformant implementation **SHOULD** expose a machine-readable configuration equivalent to:

```

qcds_fabric:
  version: "1.0"
  layer_id: "L0"

  base_bundle:
    width_B: 8
    bundles_G: 64
    binary: true

  oracle_topology:
    mode: replicated_per_channel
    active_stack_id: dna_stack_v1

```

```

immutable_within_comparison_bank: true

rotations:
  dimension_null:
    enabled: true
    full_coverage: true
  positional:
    enabled: false
    maps: []
  oracle_exposure:
    enabled: false
    maps: []

local_kernel:
  substrate: hybrid
  amplification: grover_style
  shots: 4096
  m_max: 40

stabilization:
  metrics:
    - normalized_lift
    - entropy
    - agreement_fraction
    - topk_jaccard
  preserve_uncertainty: true
  auto_prune: false

funnel:
  next_bundle_count: 8
  preserve_provenance: true

```

Numeric values above are reference demonstrator values only.

15. Data structures / interfaces

A reference implementation SHOULD expose equivalent objects.

15.1 `BaseBundle`

Required fields:

- bundle identifier,
- ordered dimension identities,
- values / state encoding,
- provenance,
- semantic domain metadata.

15.2 `ChannelView`

Required fields:

- base bundle identifier,
- null dimension identifier or `none`,
- position map,
- oracle map,
- active oracle stack version,
- substrate target,
- transformation provenance.

15.3 `TruthDistribution`

Required fields SHOULD include:

- state / hypothesis support,
- probabilities / scores,
- normalization information,
- top-K,
- entropy,
- oracle agreement / contradiction markers,
- amplification / iteration provenance.

15.4 `StabilizedReturn`

Required fields:

- stabilized distribution / representation,
- per-dimension influence diagnostics,
- position / oracle sensitivity where measured,
- retained uncertainty,
- comparison metrics,
- pruning / damping actions,
- provenance.

15.5 `Syntract`

A bound structure that may include overlapping dimensions, relations, processes, states, time, evidence, constraints, contradictions, and higher-order compositions. A Syntract MAY be used as a later dimension.

16. Validation and falsification requirements

QCDS Fabric v1 is intended to be experimentally testable.

16.1 Required ablation families

A serious implementation SHOULD compare at minimum:

1. local QCDS without fabric,
2. parallel fabric without null diagnostics,
3. dimension nulling without stabilization,
4. dimension nulling + stabilization,
5. positional rotation on / off,
6. oracle rotation on / off where applicable,
7. recursive funnel on / off,
8. full fabric.

16.2 Dimensional-noise test

Inject controlled dimension-specific noise or bias. Measure whether the null bank identifies the affected dimension and whether stabilization improves truth-distribution recovery compared with the no-nulling baseline.

16.3 Position-bias test

Inject deterministic position / slot bias. Verify that positional rotation plus inverse mapping reduces orientation sensitivity relative to a fixed orientation.

16.4 Oracle-bias test

Inject a misweighted, unstable, or position-sensitive oracle. Verify that oracle exposure rotation detects abnormal sensitivity without falsely attributing it to the input dimension.

16.5 Funnel-preservation test

Compare flat inference against multi-layer funnel inference. The funnel SHOULD preserve or improve relevant truth structure and uncertainty. If the funnel systematically destroys signal, the compile geometry or binding operator must be revised.

16.6 Cross-substrate equivalence

On bounded reference problems, CPU / simulator / QPU implementations SHOULD agree within expected numerical / hardware-noise tolerances after canonical coordinate mapping.

16.7 Falsification triggers

The following outcomes require revision of claims or mechanisms:

- null diagnostics do not outperform matched additional-sampling baselines,
- stabilization suppresses true signal as often as injected noise,
- recursive funneling loses essential information compared with flat controls,
- oracle rotation cannot distinguish oracle bias from dimension bias,
- logical-space scaling is reported by counting derived views as independent dimensions,
- cross-substrate results diverge beyond explained noise / approximation error.

17. Recommended metrics

The fabric does not canonically require one scalar truth metric. A validation suite SHOULD expose a vector of diagnostics.

Recommended metrics include:

- union probability,
- normalized lift versus local baseline,
- entropy H ,
- oracle agreement fraction,
- Top-K Jaccard,
- cross-view agreement,
- orientation sensitivity,
- null-dimension sensitivity,
- oracle sensitivity,
- contradiction persistence,
- phase spread / m tolerance where relevant,
- provenance-complete stability across cycles.

A peak is not “truth” merely because it is high. Truth-Alignment requires stability against evidence, contradiction, and repeated inference.

18. Reference funnel semantics

18.1 Contraction

N → ... → 1

18.2 Expansion

```
1 → ... → N
```

Used for discovery: hold a bound requirement or Syntract fixed and open compatible possibilities / mechanisms / experiments around it.

18.3 Bidirectional recursion

A conformant system MAY alternate contraction and expansion:

```
contract → bind → expand → test → rebind → contract
```

The four-phase architecture remains unchanged.

19. Oracle governance and self-evolution

Oracles are powerful because they define what shapes the Condition space. Therefore:

- every oracle MUST have identity and provenance,
- externally grounded oracles SHOULD link to measurement / evidence / rules / validated models,
- oracle changes MUST be versioned,
- oracle evolution SHOULD occur between comparison banks,
- candidate oracles MAY be generated by humans, agents, LLMs, or later QCDS cycles,
- no oracle becomes “true” because QCDS generated it; it must survive evidence and outcome checks,
- oracle rotation MAY be used to detect dominance, fragility, or implementation bias.

20. Safety and truth-alignment boundary

Internal coherence is not automatically external truth. In scientific, clinical, physical-control, or safety-critical domains, Truth-Alignment SHOULD include validated measurement, experiment, causal checks, safety constraints, and real outcomes.

A conformant QCDS Fabric implementation MUST distinguish:

- internal distribution coherence,
- evidence support,
- external validation,
- operational action authorization.

The architecture is designed so that new evidence can weaken or eliminate a previously strong bound state.

21. DNA as a reference stress-test, not a special-case architecture

DNA is a useful stress-test because the stored sequence is not the full biological logical space. Relevant dimensions can include variant, regulation, expression, protein structure, pathway interaction, cell state, tissue, environment, phenotype, development, aging, experiment, and causal intervention.

QCDS Fabric treats these as a potentially very wide set of independent and higher-order dimensions that can be compiled into bounded local bundles, compared under repeated oracle-constrained views, stabilized, and recursively bound.

The same fabric topology applies to other high-dimensional domains such as sensor fusion, autonomous systems, plasma / tokamak control, materials, and scientific discovery. The domain changes the Conditions and oracles; it does not redefine the QCDS core.

22. Conformance checklist

An implementation claiming **QCDS Fabric v1 conformant** MUST satisfy all of the following:

- ☐ Four-phase QCDS core is explicit.
- ☐ Local pass is inference-first and distribution-bearing.
- ☐ Same active oracle stack is replicated before every channel in a core comparison bank.
- ☐ Full core null bank nulls a different dimension in each view.
- ☐ `∅` is distinct from `0` and `?`.
- ☐ View provenance records null / position / oracle maps.
- ☐ Derived rotational views are not counted as independent dimensions by default.
- ☐ Physical quantum rotation is distinguished from architectural rotations.
- ☐ Stabilization exposes uncertainty and diagnostic deltas.
- ☐ Recursive funnel receives stabilized returns, not untracked raw outputs.
- ☐ Layer widths are configurable.
- ☐ Syntract Bind retains evidence / contradiction / composition provenance.
- ☐ Classical or quantum substrate implementation preserves the same logical semantics.
- ☐ Claims of bias suppression are measurable against baselines.

23. Canonical short form

When the full specification is not available, QCDS Fabric v1 may be summarized as:

> Conditions open the space. The same oracle regime interrogates every parallel channel. Rotational Dimension Nulling removes a different dimension in each view; optional positional and oracle rotations test other bias axes. Local QCDS passes return truth distributions. Those distributions are compared and stabilized before entering a recursive serial funnel. Higher-order returns are rebound until a Syntract emerges. The fabric width is configurable, the local kernel remains bounded, and independent logical dimensions must never be confused with derived diagnostic views.

Canonical mnemonic:

```

CONDITIONS
↓
ORACLES PER CHANNEL
↓
PARALLEL VIEWS
↓
NULL / ROTATE / INFER
↓
STABILIZE
↓
RECURSIVE FUNNEL
↓
SYNTRACT BIND
↻

```

24. Versioning rules

24.1 What requires a major version

A change requires QCDS Fabric v2 or later if it changes any of:

- the four-phase architectural identity,

- null semantics,
- oracle-per-channel comparison topology,
- independent-dimension logical-space accounting,
- the requirement that stabilization precede recursive higher-order funnel binding,
- Syntract Binding as the truth-alignment output structure.

24.2 What may remain v1.x

The following can evolve in compatible v1.x releases:

- specific stabilization metrics,
- additional rotation modes,
- compile heuristics,
- substrate implementations,
- binding operators,
- oracle libraries,
- benchmark suites,
- performance optimizations.

25. Provenance and relation to earlier QCDS material

This specification consolidates and sharpens the published / working QCDS architecture:

- the four-phase QCDS core,
- exact / mask / weighted oracle semantics,
- Grover-style amplification inside Recursive Inference,
- parallel ingress and sequential funneling,
- uncertainty-bearing TruthDistributions,
- positional rotational return / bias reduction,
- Syntract Binding and recursive re-entry.

QCDS Fabric v1 additionally locks the clarified fabric semantics formalized in August 2026:

- same oracle regime before every channel,
- Rotational Dimension Nulling as a core comparison bank,
- strict `null ≠ 0 ≠ ?` semantics,
- oracle exposure rotation as a canonical diagnostic type,
- separation of independent dimensions from derived execution views,
- recursive funneling over stabilized returns.

Primary references

1. Patrik Sundblom, **"The Syntract Vision"**, 2026, CC BY 4.0. Zenodo record 22031525, DOI 10.5281/zenodo.22031525.
2. Patrik Sundblom, **"QCDS Case Atlas"**, 2026 — especially QCDS Core, Amplification, Optional Inference Composition, Rotational Return / Bias, Logical Space, and deepening sections.
3. Patrik Sundblom, **"Inference Is All You Need — QCDS"**, Zenodo record 15455541.
4. QCDS reference code / working implementation, GitHub: ``github.com/iampathat/QCDS``.

26. Attribution

Original QCDS / Syntract architecture and theory: Patrik Sundblom.

QCDS FABRIC v1.0 / CANONICAL TECHNICAL SPECIFICATION

This specification is licensed under **CC BY 4.0**. Implementations may be licensed separately; the reference QCDS code lineage uses the MIT license unless otherwise stated.

CANONICAL LOCK

QCDS FABRIC v1.0

The canonical architecture is the four-phase QCDS core composed as a scalable inference fabric: same oracle regime per comparison channel; a different null dimension per core rotational view; optional positional and oracle rotations; distribution-preserving stabilization; recursive serial funneling; Syntract Binding; strict separation between independent logical dimensions and derived diagnostic views.

Patrik Sundblom

QCDS / The Syntract Vision